Mithril in Game-based Syntax

Wuyunsiqin[1], Bingsheng Zhang[1], Markulf Kohlweiss[2]
[1]Zhejiang University, {oyun, bingsheng}@zju.edu.cn
[2]University of Edinburgh, mkohlwei@ed.ac.uk

December 7, 2025

1 Preliminaries

1.1 Notations

There are some basic notation and conventions used throughout this work.

- The security parameter is denoted by $\lambda \in \mathbb{N}$.
- For any finite set S, we write $|S|$ for its cardinality. We use the notation

  $$x \xleftarrow{\$} S$$

  to indicate that x is sampled uniformly at random from the finite set S.
- A function $\mathsf{negl} : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is called *negligible* if for every polynomial $p(\cdot)$ there exists $N \in \mathbb{N}$ such that for all $\lambda \geq N$ it holds that $\mathsf{negl}(\lambda) < 1/p(\lambda)$. We write $\mathsf{negl}(\lambda)$ to denote an unspecified negligible function in the security parameter λ.

1.2 BLS Signature Scheme

Let λ be the security parameter. We assume a bilinear group generation algorithm which, on input 1^λ, outputs

$$(q, G_1, G_2, G_T, g_1, g_2, e),$$

where G_1, G_2, G_T are cyclic groups of prime order q, $g_1 \in G_1$, $g_2 \in G_2$ are generators, and $e : G_1 \times G_2 \to G_T$ is a bilinear pairing. Let $H : \{0,1\}^* \to G_1$ be a hash function modeled as a random oracle.

We write the BLS signature scheme as

$$\Pi_{\mathsf{BLS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Sign}, \mathsf{Ver}).$$

- $pp \leftarrow \mathsf{Setup}(1^\lambda)$: run the group generator to obtain $(q, G_1, G_2, G_T, g_1, g_2, e)$ and fix H. Output public parameters

$$\mathsf{pp} = (q, G_1, G_2, G_T, g_1, g_2, e, H).$$

- $(sk, vk) \leftarrow \mathsf{KGen}(\mathsf{pp})$: sample a secret key $sk \xleftarrow{\$} \mathbb{Z}_q$, where $\mathbb{Z}_q$ denotes the integers modulo q, and set

$$vk = g_2^{sk} \in G_2.$$

  Output the key pair (sk, vk).

- $\sigma \leftarrow \mathsf{Sign}(\mathsf{pp}, sk, msg)$: on input a message $msg \in \{0,1\}^*$, compute $h \leftarrow H(msg) \in G_1$ and set

$$\sigma = h^{sk} \in G_1.$$

  Output the signature σ.

- $b \leftarrow \mathsf{Ver}(\mathsf{pp}, vk, msg, \sigma)$, where $b \in \{0,1\}$: compute $h \leftarrow H(msg)$ and output

$$b = 1 \text{ iff } e(\sigma, g_2) = e(h, vk).$$

1.3 Merkle Tree

Let $H_{\mathsf{MT}} : \{0,1\}^* \to \{0,1\}^\lambda$ be a collision-resistant hash function.

We denote the Merkle tree primitive by

$$\Pi_{\mathsf{MT}} = (\mathsf{Create}, \mathsf{Check}).$$

- $\mathsf{avk} \leftarrow \mathsf{Create}(\mathbf{vk}, \mathbf{w})$: on input a vector of verification keys $\mathbf{vk} = (vk_1, \ldots, vk_N)$ and a matching stake vector $\mathbf{w} = (w_1, \ldots, w_N)$, where $N \in \mathbb{N}$ denotes the number of parties, set leaves $\ell_i := H_{\mathsf{MT}}(vk_i, w_i)$ and output the Merkle root avk, which we also call an aggregate verification key.

- $b \leftarrow \mathsf{Check}(\mathsf{avk}, vk, w, path)$, where $b \in \{0,1\}$: on input a root avk, a pair (vk, w), and an authentication path $path$, recompute the root from the leaf $\ell := H_{\mathsf{MT}}(vk, w)$ and $path$. Output $b = 1$ iff the recomputed root equals avk, and $b = 0$ otherwise.

2 Stake-based threshold multi-signature

2.1 Syntax

We consider a stake-based threshold multi-signature (STMS) scheme whose acceptance is determined on a per-stake basis via a "t wins in n lotteries" parametrization, for some integers $t, n \in \mathbb{N}$.

$$\Pi_{\mathsf{STMS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer}).$$

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$: on input a security parameter 1^λ, outputs public system parameters pp. These parameters are implicit inputs to all other algorithms.

- $(sk, vk) \leftarrow \mathsf{KGen}(\mathsf{pp})$: on input the public parameters pp, generates a signing/verification key pair (sk, vk) for a single party.

- $\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w})$: given a list of verification keys $\mathbf{vk} := (vk_1, \ldots, vk_N)$ and a matching stake vector $\mathbf{w} := (w_1, \ldots, w_N)$, where $N \in \mathbb{N}$ denotes the number of parties, deterministically produces an aggregate verification key avk.

- $b \leftarrow \mathsf{ACheck}(\mathsf{avk}, \mathbf{vk}, \mathbf{w})$, where $b \in \{0, 1\}$: deterministically checks whether avk is a valid aggregate verification key for $(\mathbf{vk}, \mathbf{w})$, outputting $b = 1$ if it accepts and $b = 0$ otherwise.

- $\sigma \leftarrow \mathsf{Sig}(vk, w, sk, j, m)$: attempts to produce a signature σ for message m and lottery index $j \in [n]$ under verification key vk, secret key sk, and stake w. It outputs $\perp$ if the user is ineligible for (m, j) under stake w.

- $b \leftarrow \mathsf{Ver}(vk, w, j, m, \sigma)$, where $b \in \{0, 1\}$: given a verification key vk, a stake value w, a lottery index j, a message m, and a signature σ, deterministically checks whether σ is a valid signature for (m, j) under (vk, w) and outputs $b = 1$ if it accepts and $b = 0$ otherwise.

- $\tau \leftarrow \mathsf{ASig}(\mathbf{vk}, \mathbf{w}, m, \{(vk_i, \sigma_i)\}_{i=1}^t)$: given $(\mathbf{vk}, \mathbf{w})$, a message m, and a multiset $\{(vk_i, \sigma_i)\}_{i=1}^t$ of signatures, attempts to produce an aggregate signature τ; it outputs $\perp$ if the provided inputs are insufficient or inconsistent.

- $b \leftarrow \mathsf{AVer}(\mathsf{avk}, m, \tau, T)$, where $b \in \{0, 1\}$: given a message m, an aggregate verification key avk, an aggregate signature τ, and a stake threshold parameter T, outputs $b = 1$ if τ is accepted as a valid aggregate (stake-based threshold) multi-signature on m under avk and threshold T, and $b = 0$ otherwise.

2.1.1 Correctness

Let

$$\Pi_{\mathsf{STMS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer})$$

be a stake-based threshold multi-signature (STMS) scheme as specified above. Let $\mathcal{M}$ denote the message space.

Definition (Correctness). We say that Π_{STMS} is *correct* if for every security parameter $\lambda \in \mathbb{N}$, every number of parties $N \in \mathbb{N}$, every vector of verification keys $\mathbf{vk} = (vk_1, \ldots, vk_N)$ generated honestly by KGen, every stake vector $\mathbf{w} = (w_1, \ldots, w_N)$, every message $m \in \mathcal{M}$, every stake threshold parameter T, and every lottery index j in the relevant index set, the following properties hold

except with negligible probability in λ over the randomness of all algorithms involved.

- **Correctness of aggregate key.** If $\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w})$, then
$$\mathsf{ACheck}(\mathsf{avk}, \mathbf{vk}, \mathbf{w}) = 1.$$
- **Correctness of individual signatures.** For every party index $i \in [N]$, let (sk_i, vk_i) be generated by $(sk_i, vk_i) \leftarrow \mathsf{KGen}(\mathsf{pp})$. If $\sigma_i \leftarrow \mathsf{Sig}(vk_i, w_i, sk_i, j, m)$ and $\sigma_i \neq \perp$, then
$$\mathsf{Ver}(vk_i, w_i, j, m, \sigma_i) = 1.$$
- **Correctness of aggregate signatures.** Let $t \in \mathbb{N}$ and, for each $i \in [t]$, let (sk_i, vk_i) be generated by $(sk_i, vk_i) \leftarrow \mathsf{KGen}(\mathsf{pp})$ and let w_i be the corresponding stake value. For a message m and a lottery index j, let $\sigma_i \leftarrow \mathsf{Sig}(vk_i, w_i, sk_i, j, m)$ be such that $\sigma_i \neq \perp$ for all $i \in [t]$. Let
$$\tau \leftarrow \mathsf{ASig}(\mathbf{vk}, \mathbf{w}, m, \{(vk_i, \sigma_i)\}_{i=1}^{t}) \quad \text{and} \quad \mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w}).$$
If the total stake of these t parties meets the threshold, i.e.,
$$\sum_{i=1}^{t} w_i \;\geq\; T,$$
then
$$\mathsf{AVer}(\mathsf{avk}, m, \tau, T) = 1.$$

2.1.2 Security

Stake-based unforgeability α-UF-CMA. In the unforgeability experiment, the challenger runs Setup, generates honest keys (sk_i, vk_i), fixes $\mathbf{w}$, and gives $(\mathsf{pp}, \mathbf{vk}, \mathbf{w})$ to the adversary $\mathcal{A}$. The adversary can:

- query *signing oracles* for any honest user i, any j, and any message m, receiving $\mathsf{Sig}(vk_i, w_i, sk_i, j, m)$;
- *corrupt* users to obtain sk_i (and then act under those keys).

Let C^* be the set of corrupted users. Let H^* be the set of honest users that *participated* on the forged topic (made at least one signing query for the challenge topic) under the forged aggregate key. The experiment *charges* only successful eligibility (i.e., $\mathsf{Sig} \neq \perp$) and enforces the stake bound
$$\sum_{i \in C^* \cup H^*} w_i \;\leq\; \alpha W.$$
Finally, $\mathcal{A}$ outputs $(\mathbf{vk}^*, \mathsf{avk}^*, m^*, \tau^*)$; it wins if
$$\mathsf{ACheck}(\mathsf{avk}^*, \mathbf{vk}^*, \mathbf{w}) = 1, \quad \mathsf{AVer}(\mathsf{avk}^*, m^*, \tau^*, W) = 1,$$
and τ^* is *fresh*, i.e., it cannot be obtained by aggregating local proofs returned (or derivable) from oracle responses. An STMS is α-UF-CMA secure if every PPT adversary wins with only negligible probability.

Strengthened unforgeability α-UF-CMA-S. As above, but the adversary may first register arbitrary public keys (and corresponding stake placement) before AKey is run (key-substitution/registration stage). The rest of the game is identical. Security requires negligible winning probability for all PPT adversaries.

2.2 Instantiation

2.2.1 Mithril

We write the Mithril scheme as

$$\Pi_{\mathsf{Mithril}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{ASig}, \mathsf{AVer}).$$

- $(\mathsf{pp}) \leftarrow \mathsf{Setup}(1^\lambda)$:
  run $\Pi_{\mathsf{BLS}}.\mathsf{Setup}(1^\lambda)$ to obtain $pp_{BLS} = (q, G_1, G_2, G_T, g_1, g_2, e, H)$
  fix a dense mapping / eligibility function
  $$\mathsf{Eval} : (msg, index, \sigma) \mapsto \{0,1\}^\lambda,$$
  a weighting function
  $$\varphi : \mathbb{N} \to [0,1],$$
  e.g. $\varphi(w) = 1 - (1-f)^w$.
  Index range
  $$\mu \in \mathbb{N},$$
  a quorum or lottery parameter
  $$k \in \{1, \ldots, \mu\}.$$
  Output
  $$\mathsf{pp} = (pp_{BLS}, \mathsf{Eval}, \varphi, \mu, k).$$
- $(sk, vk) \leftarrow \mathsf{KGen}(\mathsf{pp})$:
  parse pp_{BLS} for BLS signature, run $\Pi_{\mathsf{BLS}}.\mathsf{KGen}(\mathsf{pp}_{\mathsf{BLS}})$
  $$(sk, vk) \leftarrow \Pi_{\mathsf{BLS}}.\mathsf{KGen}(\mathsf{pp}_{\mathsf{BLS}})$$
  Output (sk, vk).
- $\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w})$:
  given
  $$\mathbf{vk} = (vk_1, \ldots, vk_N),$$
  $$\mathbf{w} = (w_1, \ldots, w_N),$$
  set
  $$\mathsf{avk} \leftarrow \Pi_{\mathsf{MT}}.\mathsf{Create}(\mathbf{vk}, \mathbf{w}).$$

- $b \leftarrow \mathsf{ACheck}(\mathsf{avk}, \mathbf{vk}, \mathbf{w})$, where $b \in \{0,1\}$:
  compute
  $$\mathsf{avk}' \leftarrow \Pi_{\mathsf{MT}}.\mathsf{Create}(\mathbf{vk}, \mathbf{w}).$$
  output b=1 iff $\mathsf{avk} = \mathsf{avk}'$.

- $\pi \leftarrow \mathsf{Sig}(\mathsf{pp}, vk, w, sk, j, m)$:
  compute the BLS signature
  $$\sigma \leftarrow \Pi_{\mathsf{BLS}}.\mathsf{Sign}(\mathsf{pp}, sk, m),$$
  compute the eligibility value
  $$ev \leftarrow \mathsf{Eval}(\sigma, j, m),$$
  and check the threshold
  $$ev \;<\; \lfloor \varphi(w) \cdot 2^{\lambda} \rfloor.$$
  If true, output
  $$\pi = (j, \sigma, vk, w).$$
  Otherwise output $\perp$.

- $\tau \leftarrow \mathsf{ASig}(\mathsf{pp}, m, \{\pi_i\}_{i=1}^{t})$:
  parse each
  $$\pi_i = (j_i, \sigma_i, vk_i, w_i),$$
  with pairwise distinct
  $$j_i \in \{1, \ldots, \mu\}.$$
  Set
  $$\sigma^{\star} \;=\; \prod_{i=1}^{t} \sigma_i \;\in\; G_1,$$
  $$vk^{\star} \;=\; \prod_{i=1}^{t} vk_i \;\in\; G_2.$$
  Derive a batching seed
  $$e_{\sigma} \;\leftarrow\; H\big(\sigma_1 \,\|\, \cdots \,\|\, \sigma_t\big),$$
  and coefficients for $i = 1, \ldots, t$
  $$\rho_i \;\leftarrow\; \mathsf{H}_{\lambda}(e_{\sigma} \,\|\, i) \in \mathbb{Z}_q^{*}.$$
  Compute the batch aggregates
  $$\mu \;=\; \prod_{i=1}^{t} \sigma_i^{\rho_i} \;\in\; G_1,$$

$$ivk \;=\; \prod_{i=1}^{t} vk_i^{\rho_i} \;\in\; G_2.$$

Output

$$\tau \;=\; (\,\sigma^\star,\, vk^\star,\, \mu,\, ivk,\, e_\sigma).$$

- $b \leftarrow \mathsf{AVer}(\mathsf{pp}, \mathsf{avk}, m, \tau)$, where $b \in \{0,1\}$:
  parse

  $$\tau = (\sigma^\star, vk^\star, \mu, ivk, e_\sigma).$$

  Check that the indices are pairwise distinct. Compute

  $$h \leftarrow H(m).$$

  Verify the equation

  $$e(\sigma^\star, g_2) \;=\; e(h, vk^\star).$$

  Recompute the batching coefficients for $i = 1, \ldots, t$

  $$\rho_i \;\leftarrow\; \mathsf{H}(e_\sigma \,\|\, i) \in \mathbb{Z}_q^*,$$

  and the corresponding aggregated public key

  $$ivk' \;=\; \prod_{i=1}^{t} vk_i^{\rho_i} \;\in\; G_2.$$

  Check consistency of the submitted aggregated key

  $$ivk' \;=\; ivk.$$

  Verify the batched equation

  $$e(\mu, g_2) \;=\; e(h, ivk).$$

  Output

  $$b \;=\; 1 \text{ iff all checks above hold.}$$

2.2.2 Security

We now formalize the security of a stake-based threshold multi-signature scheme in the standard game-based style. Intuitively, an adversary should not be able to produce an aggregate signature that passes verification unless all the participating stake either (i) belongs to corrupted parties, or (ii) comes from honest parties on message–lottery pairs for which the adversary has obtained valid individual signatures.

Adversarial capabilities. We allow the adversary to corrupt an arbitrary subset of users and to obtain signatures on adaptively chosen message–lottery pairs for honest users.

Experiment. Let $\Pi_{\mathsf{STMS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer})$ be an STMS scheme as above, and let $\mathcal{A}$ be a probabilistic polynomial-time (PPT) adversary. We define the experiment

$$\mathsf{Exp}^{\mathsf{stms\text{-}euf}}_{\Pi_{\mathsf{STMS}},\mathcal{A}}(\lambda)$$

as follows.

1. (*Setup.*) The challenger runs $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$. It fixes a number of users $N \in \mathbb{N}$ (polynomial in λ) and for each $i \in [N]$ samples a key pair $(\mathsf{sk}_i, \mathsf{vk}_i) \leftarrow \mathsf{KGen}(\mathsf{pp})$. It then samples a stake vector $\mathbf{w} := (w_1, \ldots, w_N) \in (\mathbb{Z}^+)^N$ and computes the aggregate verification key $\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w})$, where $\mathbf{vk} := (\mathsf{vk}_1, \ldots, \mathsf{vk}_N)$. The challenger gives $(\mathsf{pp}, N, \mathbf{vk}, \mathbf{w}, \mathsf{avk})$ to $\mathcal{A}$.

   It initializes an empty set $\mathcal{C} \subseteq [N]$ of corrupted users and an empty set $\mathcal{Q} \subseteq [N] \times [n] \times \mathcal{M}$ to record signing queries.

2. (*Oracle access.*) The adversary $\mathcal{A}$ may make adaptive oracle queries of the following types.

   - **Corruption oracle** $\mathsf{Corrupt}(i)$: on input an index $i \in [N]$, if $i \notin \mathcal{C}$ the challenger adds i to $\mathcal{C}$ and returns sk_i to $\mathcal{A}$. (Once a user is corrupted, the adversary can locally compute all its signatures and thus no signing oracle is needed for it.)
   - **Signing oracle** $\mathsf{Sign}(i, j, m)$: on input a user index $i \in [N]$, a lottery index $j \in [n]$, and a message $m \in \mathcal{M}$, if $i \notin \mathcal{C}$ the challenger computes

     $$\sigma \leftarrow \mathsf{Sig}(\mathbf{vk}, \mathbf{w}, \mathsf{sk}_i, j, m)$$

     and returns σ to $\mathcal{A}$. If $\sigma \neq \perp$, the challenger also records (i, j, m) in the set $\mathcal{Q}$. (If $i \in \mathcal{C}$, the challenger returns $\perp$ since $\mathcal{A}$ already knows sk_i.)

   The above interaction may continue for an arbitrary (polynomially bounded) number of rounds.

3. (*Output.*) Eventually, $\mathcal{A}$ outputs a tuple (m^*, τ^*, T^*), where $m^* \in \mathcal{M}$ is a message, τ^* is an alleged aggregate signature, and $T^* \in \mathbb{N}$ is a stake threshold.

   Let $b \leftarrow \mathsf{AVer}(\mathsf{avk}, m^*, \tau^*, T^*)$. If $b = 0$, the experiment outputs 0 (the adversary loses).

   Otherwise, we consider the (implicit) set of signers and lottery indices that underlie τ^*. Formally, we assume that there exists a uniquely determined multiset

   $$\mathsf{Signers}(\tau^*) \subseteq [N] \times [n]$$

   such that, informally, for each $(i, j) \in \mathsf{Signers}(\tau^*)$, the contribution of user i in lottery j is authenticated inside τ^* and is necessary for AVer to accept.

   The adversary $\mathcal{A}$ is said to *win* if both conditions hold:

(a) $\mathsf{AVer}(\mathsf{avk}, m^*, \tau^*, T^*) = 1$, and

(b) there exists a pair $(i^*, j^*) \in \mathsf{Signers}(\tau^*)$ such that $i^* \notin \mathcal{C}$ and $(i^*, j^*, m^*) \notin \mathcal{Q}$.

In this case the experiment outputs 1; otherwise it outputs 0.

We define the *stake-based threshold unforgeability advantage* of $\mathcal{A}$ against Π_{STMS} as

$$\mathsf{Adv}^{\mathsf{stms\text{-}euf}}_{\Pi_{\mathsf{STMS}},\mathcal{A}}(\lambda) := \Pr\left[\mathsf{Exp}^{\mathsf{stms\text{-}euf}}_{\Pi_{\mathsf{STMS}},\mathcal{A}}(\lambda) = 1\right],$$

where the probability is taken over the randomness of the challenger and $\mathcal{A}$.

Definition (Stake-based threshold unforgeability). We say that Π_{STMS} is *stake-based threshold existentially unforgeable under chosen-message attacks* (STMS-EUF-CMA secure) if for every PPT adversary $\mathcal{A}$ there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\mathsf{Adv}^{\mathsf{stms\text{-}euf}}_{\Pi_{\mathsf{STMS}},\mathcal{A}}(\lambda) \leq \mathsf{negl}(\lambda) \quad \text{for all } \lambda \in \mathbb{N}.$$

2.2.3 Security

Stake-based unforgeability α-UF-CMA. Let $\Pi_{\mathsf{STMS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer})$ be an STMS scheme as specified above, and let $\mathcal{A}$ be a probabilistic polynomial-time (PPT) adversary. For a fixed parameter $\alpha \in (0, 1)$ we define the following experiment

$$Exp^{\alpha\text{-}\mathsf{UF\text{-}CMA}}_{\Pi_{\mathsf{STMS}},\mathcal{A}}(\lambda)$$

and say that $\mathcal{A}$ *wins* if the experiment outputs 1.

The unforgeability experiment.

1. **Initialization.** The challenger runs $(\mathsf{pp}, m) \leftarrow \mathsf{Setup}(1^\lambda)$, where $m \in \mathbb{N}$ is the number of lotteries. It initializes:

   - arrays $(\mathsf{sk}[i])_{i\in\mathbb{N}}$ and $(\mathsf{vk}[i])_{i\in\mathbb{N}}$ with all entries set to $\perp$;
   - three disjoint index sets $\mathcal{H}, \mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{N}$, initially all empty, where $\mathcal{H}$ denotes honest parties and $\mathcal{C}_1, \mathcal{C}_2$ will correspond to parties corrupted before and after the first signing query;
   - a flag $\mathsf{SigningStarted} \leftarrow 0$;
   - associative arrays $\mathsf{Sigs}[i, m', \mathsf{vk}]$, $\mathcal{I}[t, \mathsf{vk}]$ and $\mathcal{J}[m', \mathsf{vk}]$, all initially empty, where t ranges over *topics*.

   The challenger gives (pp, m) to $\mathcal{A}$.

2. **Stake vector.** At some point $\mathcal{A}$ calls the oracle $\mathsf{SetW}(\mathbf{w})$ once, where $\mathbf{w} = (w_1, \ldots, w_N) \in (\mathbb{Z}^+)^N$ is a stake vector of arbitrary (polynomial) length N. The challenger records $\mathbf{w}$ and computes the total stake

$$w := \sum_{i=1}^{N} w_i.$$

(We assume all subsequent indices i queried by $\mathcal{A}$ satisfy $1 \leq i \leq N$.)

3. **Oracle access.** The adversary may adaptively interact with the following oracles.

   - *Key generation* $\mathsf{Gen}(i)$: if $1 \leq i \leq N$ and $\mathsf{vk}[i] = \perp$, the challenger samples $(\mathsf{sk}[i], \mathsf{vk}[i]) \leftarrow \mathsf{KGen}(\mathsf{pp})$ and sets $\mathcal{H} \leftarrow \mathcal{H} \cup \{i\}$. It returns $\mathsf{vk}[i]$.
   - *Key setting* $\mathsf{SetKey}(i, \mathsf{vk}^*)$: if $1 \leq i \leq N$, the challenger sets $\mathsf{vk}[i] \leftarrow \mathsf{vk}^*$, updates $\mathcal{C}_1 \leftarrow \mathcal{C}_1 \cup \{i\}$ and $\mathcal{H} \leftarrow \mathcal{H} \setminus \{i\}$, and returns ok. (This models adversarially chosen public keys.)
   - *Corruption* $\mathsf{Corrupt}(i)$: if $1 \leq i \leq N$, then

     $$\text{if } \mathsf{SigningStarted} = 1 \text{ set } \mathcal{C}_2 \leftarrow \mathcal{C}_2 \cup \{i\}, \quad \text{else set } \mathcal{C}_1 \leftarrow \mathcal{C}_1 \cup \{i\}.$$

     In any case update $\mathcal{H} \leftarrow \mathcal{H} \setminus \{i\}$ and return $\mathsf{sk}[i]$ to $\mathcal{A}$.
   - *Signing* $\mathsf{Sign}(i, m')$: on input a user index i and a message m', the challenger first checks that $i \in \mathcal{H}$; otherwise it returns $\perp$. It then sets $\mathsf{SigningStarted} \leftarrow 1$ and, for every lottery index $j \in [m] := \{1, \ldots, m\}$, computes

     $$\sigma \leftarrow \mathsf{Sig}(\mathbf{vk}, \mathbf{w}, \mathsf{sk}[i], j, m')$$

     and updates

     $$\mathsf{Sigs}[i, m', \mathbf{vk}] \leftarrow \mathsf{Sigs}[i, m', \mathbf{vk}] \cup \{(\mathsf{vk}[i], \sigma)\}.$$

     Let $\mathsf{topic}(m')$ denote the topic of m' (as determined by the application). The challenger also records

     $$\mathcal{I}[\mathsf{topic}(m'), \mathbf{vk}] \leftarrow \mathcal{I}[\mathsf{topic}(m'), \mathbf{vk}] \cup \{i\}, \qquad \mathcal{J}[m', \mathbf{vk}] \leftarrow \mathcal{J}[m', \mathbf{vk}] \cup \{i\}.$$

     Finally it returns the multiset $\mathsf{Sigs}[i, m', \mathbf{vk}]$ to $\mathcal{A}$.

   The above interaction may continue for an arbitrary (polynomially bounded) number of queries.

4. **Forgery.** Eventually $\mathcal{A}$ outputs a tuple $(\mathbf{vk}^*, \mathsf{avk}^*, m^*, \tau^*)$, where $\mathbf{vk}^* = (\mathsf{vk}_1^*, \ldots, \mathsf{vk}_N^*)$ is a vector of verification keys, avk^* is an aggregate verification key, m^* is a message and τ^* is an aggregate signature.

   The challenger now defines the following sets (corresponding to lines 31–32 in Game 5):

2.2.4 Security

Stake-based unforgeability α-UF-CMA.
Let $\Pi_{\mathsf{STMS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer})$ be an STMS scheme as above, and consider the following experiment for an adversary $\mathcal{A}$.

The unforgeability experiment $\mathsf{Exp}^{\alpha\text{-UF-CMA}}_{\Pi_{\mathsf{STMS}},\mathcal{A}}(\lambda)$.

1. (*Setup.*) The challenger runs $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, fixes a number of users $N \in \mathbb{N}$, and for each $i \in [N]$ samples a key pair $(\mathsf{sk}_i, \mathsf{vk}_i) \leftarrow \mathsf{KGen}(\mathsf{pp})$.

   It then fixes a stake vector $\mathbf{w} := (w_1, \ldots, w_N) \in (\mathbb{Z}^+)^N$ and computes the aggregate verification key $\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w})$, where $\mathbf{vk} := (\mathsf{vk}_1, \ldots, \mathsf{vk}_N)$. It also fixes the total number of lottery indices $n \in \mathbb{N}$ and the threshold $k \in \mathbb{N}$.

   The challenger computes the total stake

$$W \;:=\; \sum_{i=1}^{N} w_i$$

   and gives $(\mathsf{pp}, N, \mathbf{vk}, \mathbf{w}, \mathsf{avk}, n, k)$ to $\mathcal{A}$. It then initializes an empty set $\mathcal{C} \subseteq [N]$ of corrupted users and an empty set $\mathcal{Q}$ to record signing queries.

2. (*Oracle access.*) The adversary $\mathcal{A}$ may make corruption and signing oracle queries, adaptively and in any order.

   - **Corruption oracle.** On input an index $i \in [N]$, if $i \notin \mathcal{C}$, the challenger adds i to $\mathcal{C}$ and returns sk_i to $\mathcal{A}$.
   - **Signing oracle.** On input a user index $i \in [N]$, a lottery index $j \in [n]$, and a message $m \in \mathcal{M}$, if $i \notin \mathcal{C}$ the challenger computes

$$\sigma \leftarrow \mathsf{Sig}(\mathbf{vk}, \mathbf{w}, \mathsf{sk}_i, j, m)$$

     and returns σ to $\mathcal{A}$. If $\sigma \neq \perp$, the challenger records (i, j, m) in the set $\mathcal{Q}$. (If $i \in \mathcal{C}$, the challenger simply returns $\perp$ since $\mathcal{A}$ already knows sk_i.)

3. (*Forgery.*) Eventually, the adversary $\mathcal{A}$ outputs a pair (m^*, τ^*), where $m^* \in \mathcal{M}$ is a message and τ^* is an alleged aggregate signature.

   Let

$$H^* \;:=\; \left\{ i \in [N] \setminus \mathcal{C} \;\middle|\; \exists j \in [n] \text{ s.t. } (i, j, m^*) \in \mathcal{Q} \right\}$$

   be the set of honest users for which $\mathcal{A}$ has obtained individual signatures on m^* via the signing oracle, and let

$$C^* \;:=\; \mathcal{C}.$$

   We define the *effective controlled stake* of $\mathcal{A}$ for m^* as

$$W^* \;:=\; \sum_{i \in C^* \cup H^*} w_i.$$

   If the stake constraint

$$W^* \;\leq\; \alpha\, W$$

   does not hold, then the experiment outputs 0 (the adversary loses).

Otherwise, the challenger computes

$$b \leftarrow \mathsf{AVer}(\mathsf{avk}, m^*, \tau^*, k),$$

where $b \in \{0,1\}$. The experiment outputs 1 (the adversary wins) if $b = 1$, and it outputs 0 otherwise.

We define the adversary's advantage in the above game by

$$\mathsf{Adv}^{\alpha\text{-UF-CMA}}_{\Pi_{\mathsf{STMS}},\mathcal{A}}(\lambda) := \Pr\Big[\mathsf{Exp}^{\alpha\text{-UF-CMA}}_{\Pi_{\mathsf{STMS}},\mathcal{A}}(\lambda) = 1\Big],$$

where the probability is taken over the randomness of the challenger and of $\mathcal{A}$.

Definition (Stake-based unforgeability). We say that Π_{STMS} is *α-UF-CMA secure* if for every PPT adversary $\mathcal{A}$ there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\mathsf{Adv}^{\alpha\text{-UF-CMA}}_{\Pi_{\mathsf{STMS}},\mathcal{A}}(\lambda) \le \mathsf{negl}(\lambda) \quad \text{for all } \lambda \in \mathbb{N}.$$

2.2.5 Security

Stake-based unforgeability α-UF-CMA. Fix a parameter $\alpha \in (0,1)$ and let $\Pi_{\mathsf{STMS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer})$ be an STMS scheme as specified above. We now define the following experiment for a PPT adversary $\mathcal{A}$.

The unforgeability experiment $\mathsf{Exp}^{\alpha\text{-}\mathsf{UF\text{-}CMA}}_{\Pi_{\mathsf{STMS}},\mathcal{A}}(\lambda)$.

1. **Initialization.** The challenger runs $(\mathsf{pp}, n) \leftarrow \mathsf{Setup}(1^\lambda)$, where n denotes the number of lotteries per message and is assumed to be part of pp. It initializes arrays of secret and verification keys $\mathsf{sk}[i] \leftarrow \perp$, $\mathsf{vk}[i] \leftarrow \perp$ for all indices i, and sets

   $$H \leftarrow \emptyset, \quad C_1 \leftarrow \emptyset, \quad C_2 \leftarrow \emptyset, \quad \mathsf{SigningStarted} \leftarrow 0.$$

   It also initializes associative maps $I[\cdot,\cdot]$ and $J[\cdot,\cdot]$ and the table $\mathsf{Sigs}[\cdot,\cdot,\cdot]$ to the empty set. Finally, it sends (pp, n) to $\mathcal{A}$.

2. **Stake vector.** At some point the adversary chooses a stake vector $\mathbf{w} = (w_1, \ldots, w_N) \in (\mathbb{Z}^+)^N$ and gives it to the challenger via a special procedure $\mathsf{SetW}(\mathbf{w})$ (this procedure may be called only once). The challenger records $\mathbf{w}$ and sets

   $$w \leftarrow \sum_{i=1}^{N} w_i$$

   (the total stake).

3. **Oracle access.** The adversary $\mathcal{A}$ is given adaptive oracle access to the following procedures. Whenever a statement of the form "$\mathsf{check}\ \Phi$" appears and Φ is false, the experiment outputs 0 and halts.

$\mathsf{Gen}(i)$**:** On input an index i,

$$\mathsf{check}\ i \leq |\mathbf{w}|.$$

If $\mathsf{vk}[i] = \perp$ then the challenger runs $(\mathsf{sk}[i], \mathsf{vk}[i]) \leftarrow \mathsf{KGen}(\mathsf{pp})$ and updates $H \leftarrow H \cup \{i\}$. It returns $\mathsf{vk}[i]$.

$\mathsf{SetKey}(i, \mathsf{vk}^*)$**:** On input an index i and a verification key vk^*,

$$\mathsf{check}\ i \leq |\mathbf{w}|.$$

The challenger sets $\mathsf{vk}[i] \leftarrow \mathsf{vk}^*$, updates $C_1 \leftarrow C_1 \cup \{i\}$ and $H \leftarrow H \setminus \{i\}$, and returns ok.

$\mathsf{Corrupt}(i)$**:** On input an index i the challenger proceeds as follows. If $\mathsf{SigningStarted} = 1$ then $C_2 \leftarrow C_2 \cup \{i\}$, otherwise $C_1 \leftarrow C_1 \cup \{i\}$. In both cases it sets $H \leftarrow H \setminus \{i\}$ and returns $\mathsf{sk}[i]$ to $\mathcal{A}$.

$\mathsf{Sign}(i, m)$**:** On input an index i and a message m,

$$\mathsf{check}\ i \in H.$$

The challenger sets $\mathsf{SigningStarted} \leftarrow 1$ and, for every lottery index $j \in [n]$, computes

$$\sigma \leftarrow \mathsf{Sig}(\mathbf{vk}, \mathbf{w}, \mathsf{sk}[i], j, m).$$

Whenever $\sigma \neq \perp$ it adds the pair $(\mathsf{vk}[i], \sigma)$ to the multiset $\mathsf{Sigs}[i, m, \mathbf{vk}]$. After the loop, it updates the index sets

$$I[m.\mathsf{topic}, \mathbf{vk}] \ \leftarrow\ I[m.\mathsf{topic}, \mathbf{vk}] \cup \{i\}, \qquad J[m, \mathbf{vk}] \ \leftarrow\ J[m, \mathbf{vk}] \cup \{i\},$$

and returns $\mathsf{Sigs}[i, m, \mathbf{vk}]$.

The interaction with these oracles may continue for an arbitrary (polynomially bounded) number of rounds.

4. **Forgery.** Eventually $\mathcal{A}$ outputs a tuple $(\mathbf{vk}^*, \mathsf{avk}^*, m^*, \tau^*)$, where $\mathbf{vk}^* = (\mathsf{vk}_1^*, \ldots, \mathsf{vk}_N^*)$ is a vector of verification keys, avk^* is an aggregate verification key, m^* is a message and τ^* is an alleged aggregate signature.

   The challenger now performs the following checks (this is the procedure α-$\mathsf{UF\text{-}CMA}$ in Game 5).

   (a) Define the set

   $$C^* \ \leftarrow\ C_1 \cup C_2 \cup \{\, i : \mathsf{vk}_i^* \neq \mathsf{vk}[i] \,\}.$$

   (b) Define

   $$H^* \ \leftarrow\ \{\, i : i \in I[m^*.\mathsf{topic}, \mathbf{vk}^*] \wedge \mathsf{vk}_i^* = \mathsf{vk}[i] \,\} \cap H.$$

(c) check

$$\sum_{i \in C^* \cup H^*} w_i \;\leq\; \alpha \cdot w.$$

(d) $\mathsf{check}\ \mathsf{ACheck}(\mathsf{avk}^*, \mathbf{vk}^*, \mathbf{w}) = 1.$

(e) Let

$$b \;\leftarrow\; \mathsf{AVer}(\mathsf{avk}^*, m^*, \tau^*, T^*),$$

where T^* is the stake threshold parameter determined by the scheme (for example, $T^* = \alpha \cdot w$ or is encoded in pp). The experiment outputs 1 iff $b = 1$, and outputs 0 otherwise.

We define the *α-stake-based unforgeability advantage* of $\mathcal{A}$ against Π_{STMS} as

$$\mathsf{Adv}^{\alpha\text{-}\mathsf{UF\text{-}CMA}}_{\Pi_{\mathsf{STMS}},\mathcal{A}}(\lambda) := \Pr\left[\mathsf{Exp}^{\alpha\text{-}\mathsf{UF\text{-}CMA}}_{\Pi_{\mathsf{STMS}},\mathcal{A}}(\lambda) = 1\right],$$

where the probability is taken over the randomness of the challenger and of $\mathcal{A}$.

We say that Π_{STMS} is *α-stake-based existentially unforgeable under chosen-message attacks* (α-$\mathsf{UF\text{-}CMA}$ secure) if for every PPT adversary $\mathcal{A}$ there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\mathsf{Adv}^{\alpha\text{-}\mathsf{UF\text{-}CMA}}_{\Pi_{\mathsf{STMS}},\mathcal{A}}(\lambda) \leq \mathsf{negl}(\lambda).$$

3 Security of the Mithril Instantiation

In this section we prove that the concrete instantiation of the stake–based threshold multi-signature (STMS) scheme given in Appendix **??** (see also [**?**]) is α-UF-CMA secure assuming the hardness of the co-CDH problem in the underlying bilinear group and the knowledge soundness of the SNARK system used for certification. The instantiation uses BLS signatures as the underlying primitive and the eligibility function as defined in [**?**]:contentReference[oaicite:1]index=1.

Theorem 3.1. *Let Π_{Mithril} be the STMS scheme defined in Appendix **??**. Assume that: (i) the BLS signature scheme is existentially unforgeable under chosen message attacks assuming the co-CDH assumption; (ii) the hash functions H and are modelled as random oracles; (iii) the adversarial stake is at most $\alpha < \frac{1}{2}$; and (iv) honest parties erase their one-epoch signing keys immediately after use. Then Π_{Mithril} satisfies α-UF-CMA security.*

Proof. Consider an adversary $\mathcal{A}$ that outputs a successful forgery $(vk^{,m^{,\tau^{)}}}$ in the STMS unforgeability experiment. By definition of STMS unforgeability, this means that: (1) $(, vk^{,w)=1}$, (2) $(, m^{,\tau^{,T)=1}}$, and (3) the total stake of the signers contributing to τ but not queried through corruption or signing oracles is at most αW.

Step 1: Reduction to a forged BLS signature. By correctness of (see Section **??**), if accepts then the aggregate signature τ contains at least k valid individual BLS signatures $\sigma_i = H(m)^{sk_i}$ coming from *distinct* verification keys vk_i. Since the adversary controls at most αW stake and k is chosen as in the Mithril lottery analysis so that the expected honest contribution exceeds the adversarial contribution with overwhelming probability, at least one of these signatures must correspond to an honest user that was neither corrupted nor queried for a signing oracle on m.

Thus, from τ we can extract an unqueried, uncorrupted BLS signature $\sigma = H(m)^{sk}$ under some public key $vk = g_2^{sk}$.

Step 2: Programming the random oracle. The reduction $\mathcal{B}$ simulating the STMS experiment embeds its co-CDH challenge (g_1^a, g_2^b) into the public key vk by setting $vk = g_2^b$ and answering all BLS signing, eligibility, and aggregation queries consistently. Since is a random oracle, $\mathcal{B}$ can ensure that the challenger's key is eligible with the correct probability $\varphi(w)$ without revealing the underlying exponent.

Step 3: Extracting the co-CDH solution. When $\mathcal{A}$ outputs its forgery, $\mathcal{B}$ obtains a valid BLS signature

$$\sigma = H(m)^b.$$

Since $\mathcal{B}$ programmed $H(m) = g_1^a$ (this happens with probability at least $1/q_H$ where q_H is the number of hash queries), we obtain:

$$\sigma = (g_1^a)^b = g_1^{ab},$$

which is a valid solution to the co-CDH problem on input (g_1^a, g_2^b).

Thus, any PPT adversary forging a Mithril aggregate signature induces a PPT adversary solving co-CDH with non-negligible probability, violating assumption (i). The only additional failure probabilities arise from: (i) the probability that all extracted signatures belong to corrupted or oracle-signing users (bounded by the adversarial stake α); (ii) randomness of the lottery eligibility tests; and (iii) the random-oracle programming. All these contribute only negligible additive terms.

Hence the scheme is α-UF-CMA secure. □